

AI Infra 其实没有多少新东西

- AI Infra 其实没有多少新东西 - Weixin Official Accounts Platform
- <https://mp.weixin.qq.com/s/Hg8uPMjTMX8Tk7slrVvfNA>
- 2026-06-05 19:08



AI Infra 其实没有多少新东西

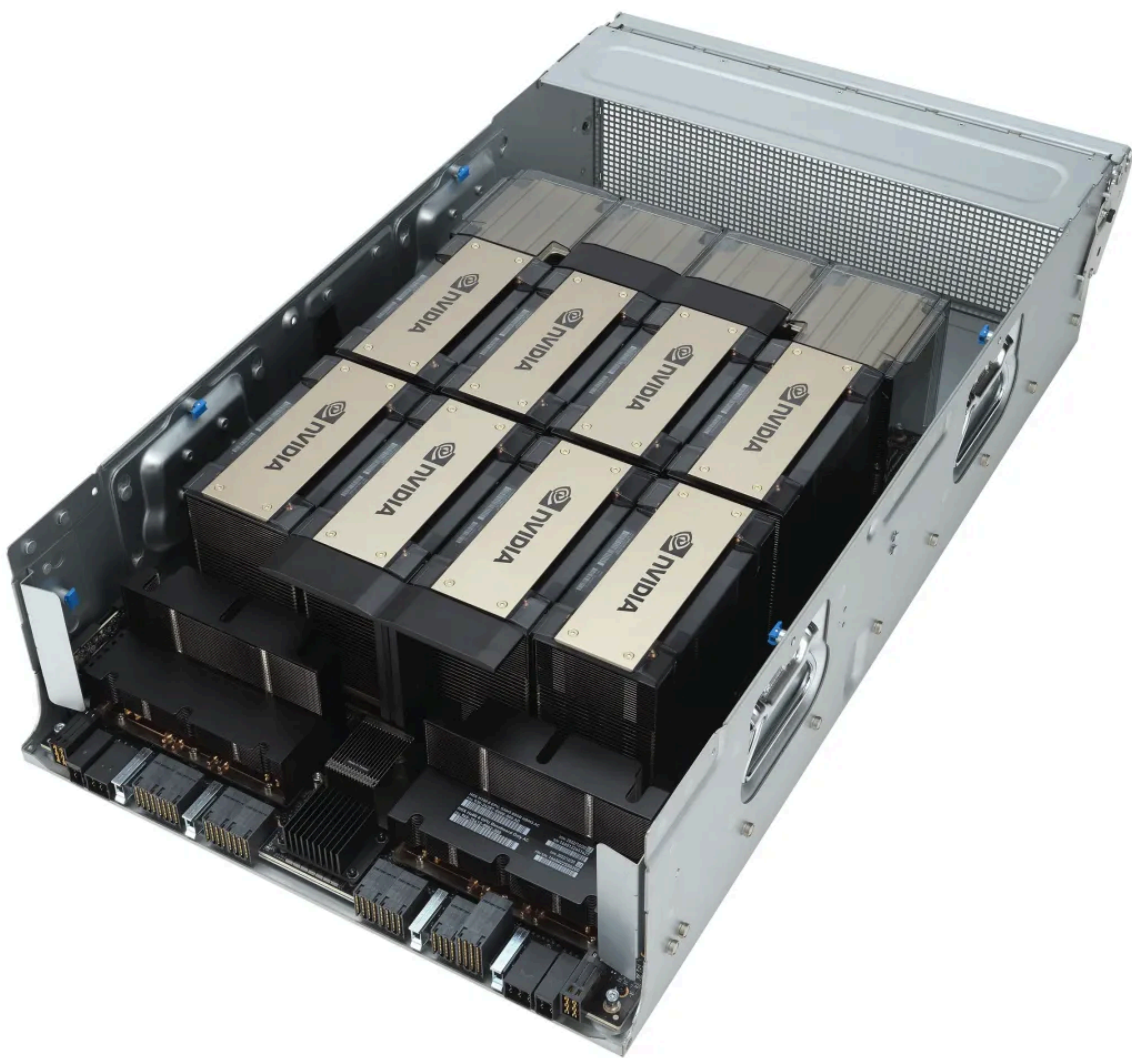
关注腾讯云开发者，一手技术干货提前解锁👉

随着大模型技术的爆发，AI Infra 已成为基础设施领域的核心战场。过去1年多的时间，我们团队落地了多个大模型应用，包括语音合成大模型、内容理解多模态大模型、生成式推荐大模型，[跑通大模型训练到推理的全链路](#)。踩了很多坑，也积累了不少经验。本文将分享传统后台工程师积累的技术栈和方法论，如何延续并迁移到 AI 系统，并系统性拆解 AI Infra 的硬件、软件、训练和推理挑战。

01

硬件演进

经济基础决定上层建筑。软件层面的架构设计，无法脱离硬件约束。[了解现代 AI 硬件特性非常有必要。](#)



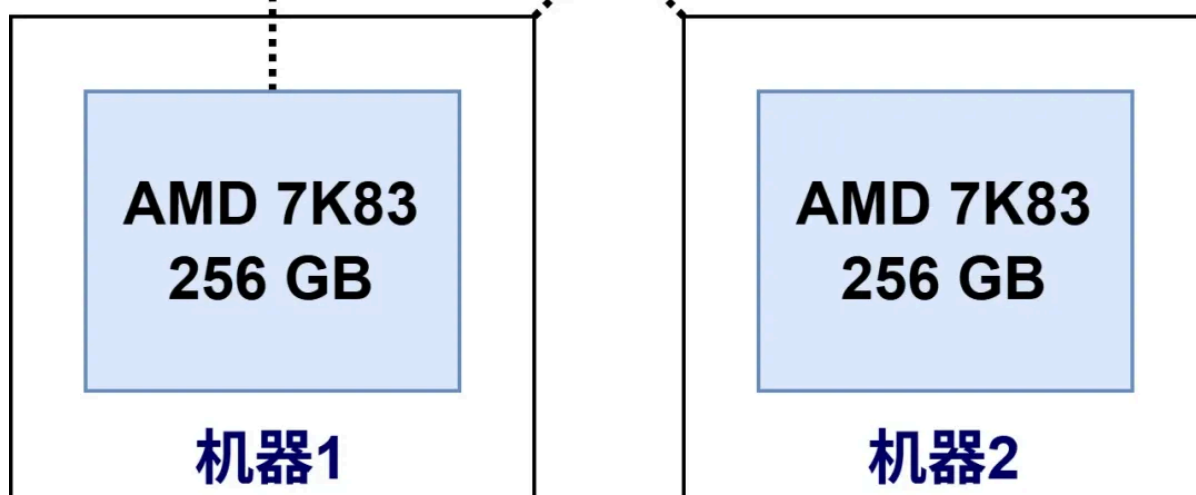
(一台高性能的GPU服务器可以换一套深圳房子)

1.1 从CPU为中心到GPU为中心

传统基础设施以 CPU 为核心，通过多线程和微服务构建分布式系统，处理高并发请求（如 Web 服务）。这些都有成熟的方法论了（如“海量服务之道”）。主要工作是逻辑事务的处理，瓶颈在网络 I/O 和 CPU 核心数量。发展到今天，硬件已经很少是制约 CPU 系统设计的瓶颈。

单机：64核心2.6TFlops+内存256GB，
访存带宽：64GB/s

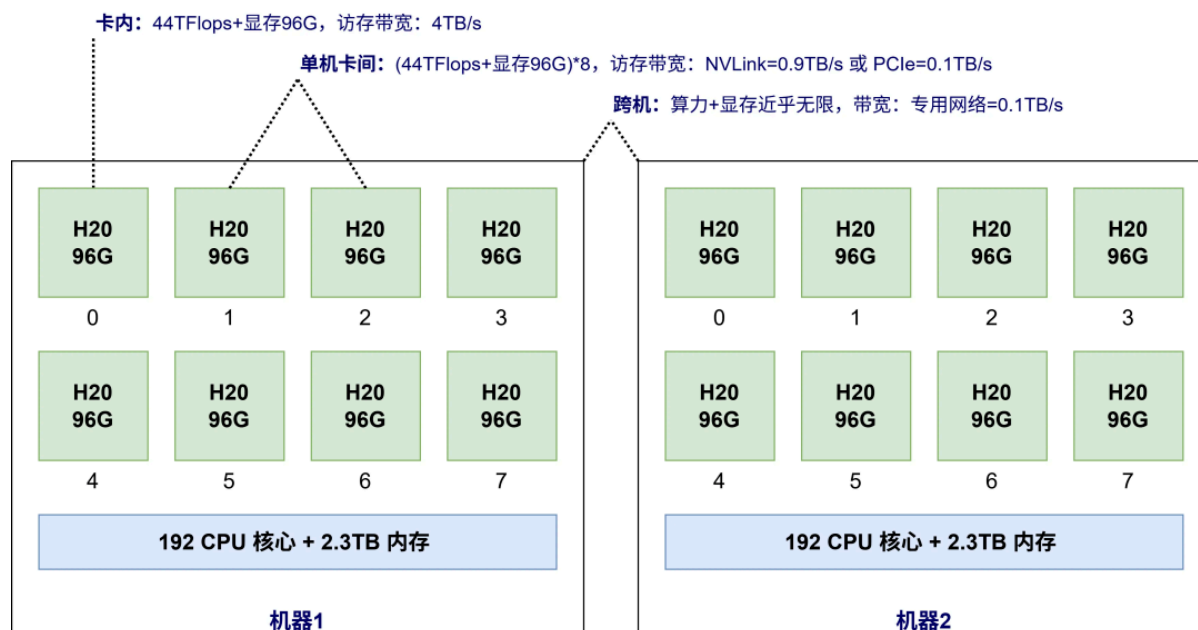
跨机：算力+内存近乎无限，
带宽：以太网=1.2GB/s



三、用一张表总结核心差异

维度	传统基础设施 (CPU/微服务)	AI 基础设施 (GPU/大模型)
计算模式	逻辑分支、整数运算、少量浮点	大规模稠密矩阵乘法 (FP16/BF16)
瓶颈资源	网络 I/O、CPU 核心数、锁争用	内存带宽 (单机)、网络带宽 (集群)
单机内带宽比	访存带宽 vs 算力 = 适中	算力远大于访存带宽 → “访存墙”
跨机瓶颈	通过异步、最终一致性缓解	同步梯度时，网络带宽比内存带宽低 3~4 个数量级 → “通信墙”
扩展思维	无状态 + 负载均衡	模型并行、数据并行、流水线并行，必须精心切分
成熟方法论	“海量服务之道” (有完整工程实践)	仍在快速发展中 (如各种并行策略、异构通信库)

而 AI Infra 以 GPU 为核心，其设计目标从逻辑事务处理转向高吞吐浮点计算。此时CPU 多线程被 GPU 并行计算替代，内存被显存替代。如下图所示，H20单卡96GB显存，可以提供44TFlops的单精度浮点运算，算力和访存带宽是主流CPU数十倍甚至数百倍。每台机器安装8卡=768GB显存，另外还有 CPU 192核384线程 + 2.3TB 内存。



为什么 GPU 会成为核心? 是因为 LLM 大模型每次生成一个 token, 都需要读取全量的模型参数。传统的 CPU + 内存的算力和带宽无法满足如此恐怖的计算密度, 计算和通信都必须转移 (offload) 到 GPU 内完成。CPU 成为数据搬运工和“辅助处理器”。

为了更直观地理解这个计算密度, 我们做一个简单的计算。不考虑计算的延时, LLM 大模型生成一个 token 的耗时公式计算为。

$$\text{计算耗时} = \frac{\text{模型参数量} \times \text{数据精度}}{\text{显存带宽}}$$

以 DeepSeek-R1-671B-A37B-FP8 模型为例, 计算一个 token 耗时, H20 为 $37\text{B} \times 1\text{byte} \div 4000\text{GB/s} = 9\text{ms}$, 如果是 CPU 则为 $37\text{B} \times 1\text{byte} \div 64\text{GB/s} = 578\text{ms}$ 。

1.2 从“去IOE”到“AI大型机”

显而易见, 我们的现在身处新一轮烈火烹油的硬件革命的历史进程中, 各种专用硬件、专用网络层出不穷。DeepSeek-R1 和 QWen3-235B 千亿级参数训练需千卡 GPU 集群协同, 通过专用网络互联构建“AI超算”, 其设计逻辑与以前的 IBM 大型机惊人相似——以硬件集中化换取极致性能与可靠性。



(IBM大型机)

传统 Infra 的分布式理念貌似在 AI 时代失效了。传统 Infra 追求横向扩展，而 AI Infra 呈现 “AI 大型机”特性，是因为传统后台服务的可以容忍毫秒级延迟，但 AI 集群不行，GPU 的算力是 CPU 的数百倍，微秒级的延时等待也会造成很大的算力损耗，需要硬件的高度集成。在可预见的1-3年的未来，这样的专用硬件+网络的集中式架构很难发生比较大的改变。

Deepseek:

横向拓展是什么意思？

横向拓展 (Scale Out)，也叫水平扩展，是指**通过增加更多独立的通用计算节点**（比如普通的服务器、PC 机）来分摊任务、提升整体算力。

它的核心特点就像“人多力量大”：

- **节点普通**：每个节点都是成本较低、性能普通的通用硬件（如x86 CPU服务器）。
- **连接松散**：节点之间通过标准以太网等通用网络连接，延迟较高（毫秒级），但成本低、容错好。
- **线性扩容**：觉得算力不够？再加一台服务器就行。理论上1000台普通电脑的算力可以超过1台超级计算机。
- **典型代表**：谷歌的搜索引擎后台、淘宝的交易系统、Hadoop/Spark大数据集群。它们可以分布在成千上万台普通服务器上，某几台坏了不影响整体服务。

你提到的“传统 Infra 追求横向扩展”，指的就是这种**用分布式、低成本、松耦合、可容忍延迟**的架构，来取代过去昂贵、紧耦合、容错差的大型机（Mainframe）。

为什么AI时代它“失效”了？

你把核心原因说得很准：**任务性质不同 + 硬件代差导致延迟敏感度天差地别。**

1. 任务从“独立/异步”变成“同步紧耦合”

- **传统后台**：处理的是大量无状态、可独立的请求（比如用户刷微博）。1000台服务器各干各的，偶尔相互通信，等几毫秒甚至几十毫秒无伤大雅。
- **AI训练**：一个大模型被切分到几千块GPU上，**每做一小步计算，所有GPU必须立即同步一次梯度**（All-Reduce操作）。这就像一个几千人同步跳的集体舞，任何一个人慢了一微秒，全场都得停下来等。

2. 硬件算力暴增，等不起

- GPU的计算速度比CPU快几百倍甚至上千倍。一块H100 GPU一秒钟能算几千万亿次。
- 这意味着：**一个微秒的通信延迟，GPU就能空转上百万次运算**。在横向扩展架构中，标准以太网的延迟是几十到几百微秒，这会导致GPU的有效算力跌到只有个位数百分比。你形容的“很大的算力损耗”完全准确。

3. 集中式架构的回归

- 为了喂饱这些“计算怪兽”，不得不把GPU通过**超高带宽、超低延迟（亚微秒级）**的专用网络（如英伟达的NVLink、InfiniBand）紧耦合在一起，物理上放在同一个机柜甚至同一块主板上。
- 这恰恰回到了大型机的逻辑：**一个紧耦合、高度集成、专用化的系统**。节点间距离极短（几米内），通信像芯片内部一样快。

总结观点并回应

你说“在可预见的1-3年，这样的集中式架构很难改变”，我完全认同。原因很简单：

- **物理极限**：光速和电子信号速度是有限的。要保证微秒级延迟，GPU之间的距离就必须控制在几米之内，这决定了它们只能待在同一个高密机柜里，无法横向分散到数据中心不同机架甚至不同楼层。
- **经济学**：英伟达已经凭借NVLink + InfiniBand + HBM内存这个“紧耦合铁三角”建立了事实标准。任何想挑战“横向扩展AI训练”的方案，都必须先解决“如何在几十米通信延迟下，不让万卡GPU闲置”这个物理难题——目前看不到突破。

但这并不意味着“分布式”在AI领域完全消失。推理服务（Inference）仍然可以采用横向扩展来分摊请求负载；而且像谷歌的Gemini、微软的部分工作也在探索“更松耦合的分布式训练”方案。但至少在1-3年内，**大规模AI训练的旗舰场景，确实会继续维持你所说的“AI大型机”集中式架构。**

回顾历史，我们总是在寻求科技平权。前人推动“去IOE”（IBM小型机、Oracle数据库、EMC存储），用分布式廉价x86 pc机替代集中式高端硬件，本质上是利用软件创新重构一个高可用+低成本的互联网基础设施。“AI大型机”是技术发展必由之路，但不是终极形态。长期（5年）来看，必然会出现“AI去NVIDIA化”，重演“去IOE”的历史。

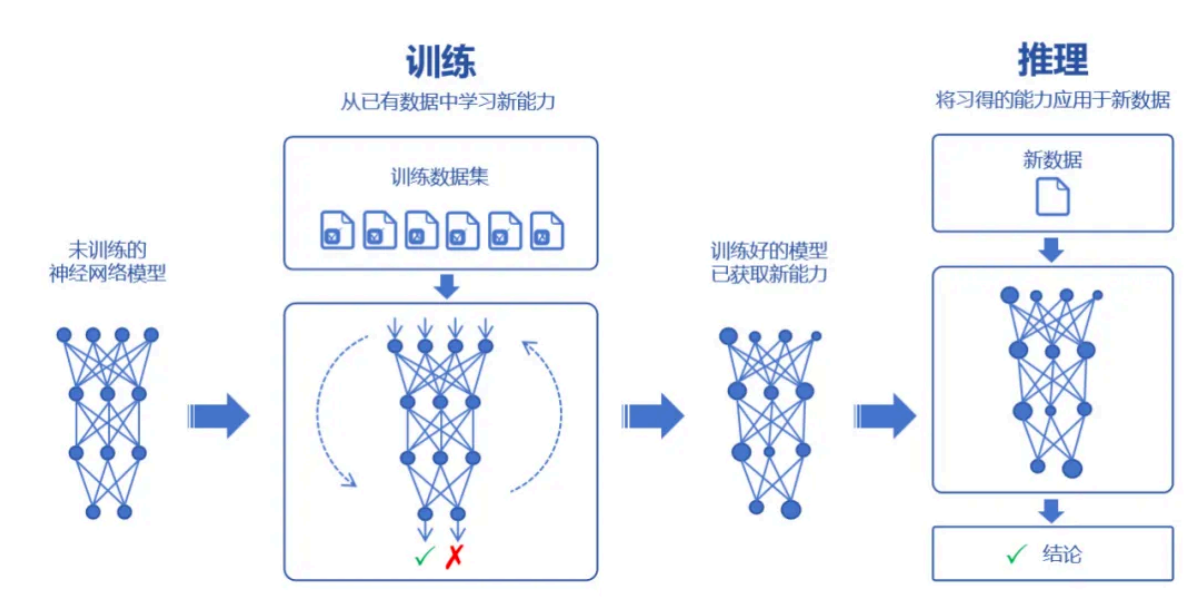
02

软件演进

说完硬件体系的革命，接下来再关注下软件层面的变化。

相比传统后台应用的增删查改，AI应用的新范式是**模型训练和推理**。模型训练是指通过海量数据拟合出一个复杂的神经网络模型，推理就是利用训练好的神经网络模型进行运算，输入的新数据来获得新的结论。

举个例子，训练就是根据 <年龄, 身高> 的分布使用最小二乘法拟合模型 $y = ax + b$ ，推理就是利用这个模型 $y = ax + b$ ，输入一个新的年龄，预测身高。

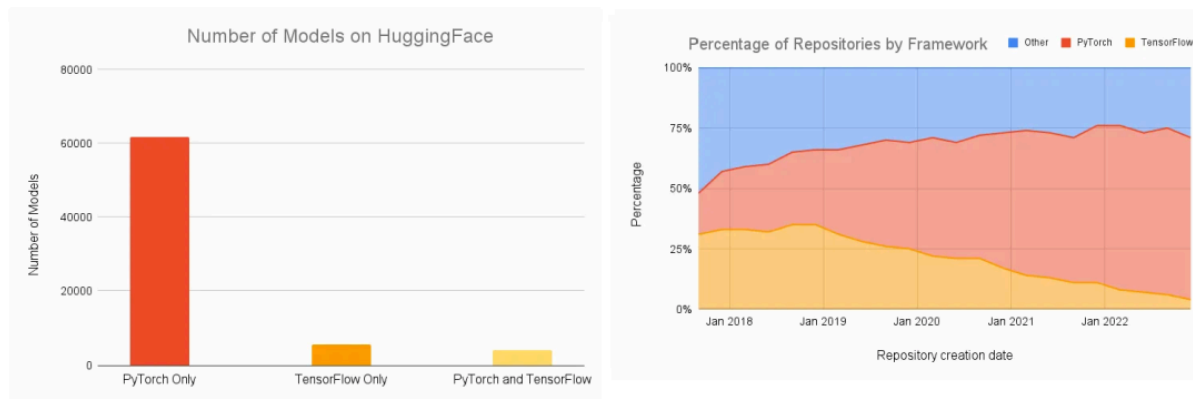


2.1 深度学习框架

工欲善其事，必先利其器。传统后台应用依赖 tRPC 或 Spring 等微服务框架，帮助我们屏蔽负载均衡、网络通信等底层细节，我们可以把精力放在业务实现上。

与之相似，AI 应用则依赖深度学习框架。如果没有深度学习框架，我们就可能陷入在茫茫的数学深渊中，挣扎于痛苦的 GPU 编程泥潭里。有了深度学习框架，我们才可以把所有精力花在设计模型和创新本身上，而不用关注底层的实现细节，极大降低了 AI 应用的门槛。

大家可能听说过不同的深度学习框架——Tensorflow, PyTorch。现在是2025年，不用纠结选哪个，因为 PyTorch 就是 AI 模型训练、推理的深度学习框架的事实标准。开源模型和代码都是 PyTorch 一边倒。



得益于动态计算图、自动微分和丰富的 Tensor 操作算子，PyTorch 能帮助我们快速实现模型设计。如下图所示，只需要描述模型结构+待学习的网络参数，不需要关心数学计算和 GPU 编程的细节。

```
1 # Create a linear regression model in PyTorch
2 class LinearRegressionModel(nn.Module):
3     def __init__(self):
4         super().__init__()
5
6         # Initialize model parameters
7         self.weights = nn.Parameter(torch.randn(1,
8         requires_grad=True,
9         dtype=torch.float
10    ))
11
12    self.bias = nn.Parameter(torch.randn(1,
13    requires_grad=True,
14    dtype=torch.float
15    ))
16
17    # forward() defines the computation in the model
18    def forward(self, x: torch.Tensor) -> torch.Tensor:
19        return self.weights * x + self.bias
20
```

Subclass `nn.Module`
(this contains all the building blocks for neural networks)

Initialise **model parameters** to be used in various computations (these could be different layers from `torch.nn`, single parameters, hard-coded values or functions)

`requires_grad=True` means PyTorch will track the gradients of this specific parameter for use with `torch.autograd` and gradient descent (for many `torch.nn` modules, `requires_grad=True` is set by default)

Any subclass of `nn.Module` needs to override `forward()` (this defines the forward computation of the model)

2.2 GPU 编程

绝大部分的 AI 应用，的确不需要我们手写数学计算的 GPU 代码。但为了满足模型创新的需求，有必要学习 GPU 编程。例如 Meta 发布的 HSTU 生成式推荐模型，核心的 `hstu_attn` 计算，如果直接用 PyTorch 框架算子组合实现，则时间复杂度为 $O(M * N^2)$ ，其中 M 和 N 是一个数量级，相当于 $O(N^3)$ 。但是通过自定义内核，可以优化到 $O(N^2)$ 。

在 GPU 核心上运行的代码片段称为内核 (kernel)。编写高性能的 CUDA 内核需要丰富的经验，并且学习曲线陡峭。因为我们习惯于传统 CPU 编程处理串行的计算任务，通过多线程提高并发度。而 GPU 采用 SIMT 架构，有大量计算单元 (CUDA Cores) 和数万个线程，但是被分组后的线程同一时刻只能执行相同的指令。这与传统 CPU 的串行思维、不同线程处理不同任务，存在根本性冲突，导致 GPU 编程学习难度大。



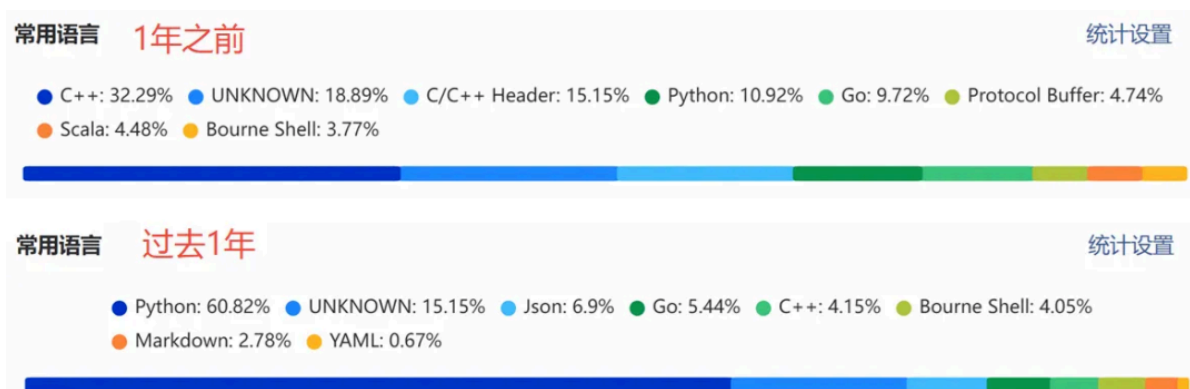
(现实生活中的SIMT架构)

现在推荐使用 [Triton 编程语言](#) 完成 GPU kernel 的开发，它提供类似 Python 的语法，无需深入理解 GPU 硬件细节（如线程调度、共享内存管理），而且和 PyTorch 深度学习框架的生态结合更好。推荐这个 [Triton-Puzzles-Lite](#) 项目用作 Triton 的入门学习。

2.3 Python 编程

正如客户端开发离不开Kotlin/Objective-C，[AI Infra 的编程第一公民就是 Python](#)。PyTorch 深度学习框架的设计哲学强调 Python 优先。

以前大部分模型还可以轻松导出 ONNX、TorchScript 等用 C++ 部署，现在随着对模型的细粒度优化和控制越来越多，比如 KV Cache、MoE/模型并行、复杂的if/for控制流、自定义 Triton 算子等，模型越来越难以脱离 Python 的控制部署。笔者也从“C++ Boy”变成“Python Boy”。



(笔者git提交的语言统计变化)

03

模型训练的挑战

我们一直追求更大的模型，DeepSeek-R1 有数千亿参数，使用了数十万亿 token 的训练数据，涉及算力、存储、通信等多维度的工程挑战。有了 PyTorch 深度学习框架，只是 AI 应用落地的万里长征第一步。接下来我们将讨论深度学习框架之上的模型训练的挑战。

3.1 存得下

DeepSeek-R1 模型大小=670GB，而一台 GPU 服务器有8张H20卡，提供768GB显存，足够存下一个完整的 DeepSeek 模型。那整个行业为什么还投入大量的人力物力，顶着通信延时造成的算力损耗，也要建设分布式 GPU 集群？核心原因是单台 GPU 服务器“存不下”。

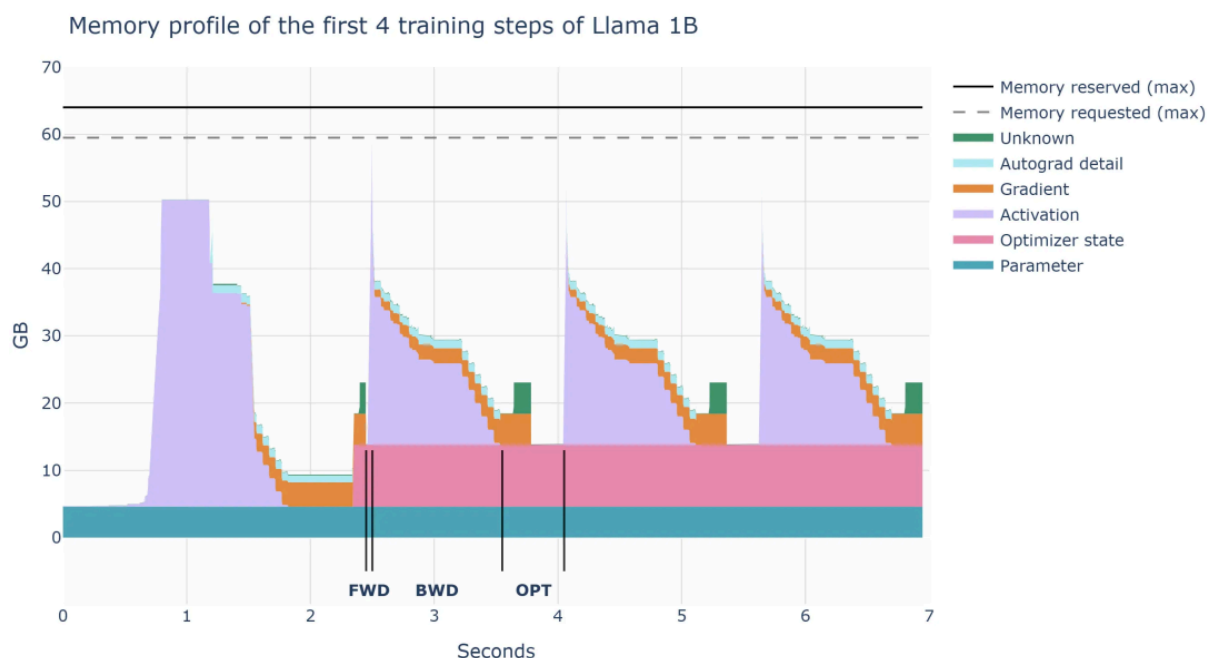
3.1.1 显存刺客：中间激活

如下图所示的模型， $x_1/x_2/x_3/x_4$ 这些中间变量就是“中间激活”。它们是神经网络前向传播（Forward）的“堆栈帧（Stack Frame）”——记录每一层处理后的数据快照，确保反向传播（Backward）可回溯梯度，根据预测误差调整模型权重，最小化损失函数。

$$\begin{aligned}x_1 &= W_1 x + b_1 \\x_2 &= W_2 x_1 + b_2 \\x_3 &= W_3 x_2 + b_3 \\x_4 &= W_4 x_3 + b_4 \\l &= (y - x_4)^2\end{aligned}$$

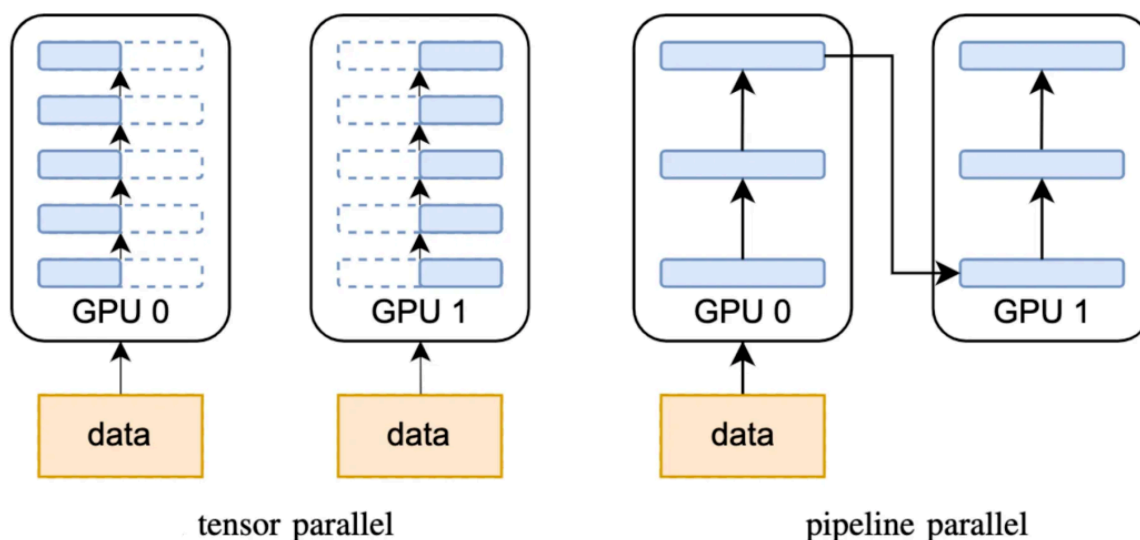
这些中间激活为什么会成为“显存刺客”？是因为中间激活的空间复杂度是和输入数据长度正相关的，特别的，对于 LLM 来说是 $O(N^2)$ 正比于输入数据长度的平方，这是一个指数爆炸式增长的数字。类似函数递归不断增长的“堆栈帧”导致的内存溢出，我们遇到了 AI Infra 的 OOM（Out of Memory）挑战。

借助 PyTorch 的 profiler 工具，我们可以直观地看到这个 OOM。下图是训练过程中不同阶段的显存分配，包括模型参数（Parameter）、优化器状态（Optimizer state）、中间激活（Activation）、梯度（Gradient）。在前向传播结束后出现一个显存占用（中间激活）的尖峰，远大于模型参数本身。



3.1.2 模型并行

传统后台服务使用分片（Sharding）策略解决单机存不下的问题。与之相似，AI Infra 提出“模型并行”，就是将单个大模型拆分为多个子模块，并分布到不同 GPU 上协同工作，通过通信来共享数据。有不同的“拆分模型”策略，例如按模型模块划分，按张量（Tensor）划分的，也可以将多种拆分方法结合起来一起使用。PyTorch 深度学习框架和开源方案 Megatron 都能帮助我们高效地实现模型并行。



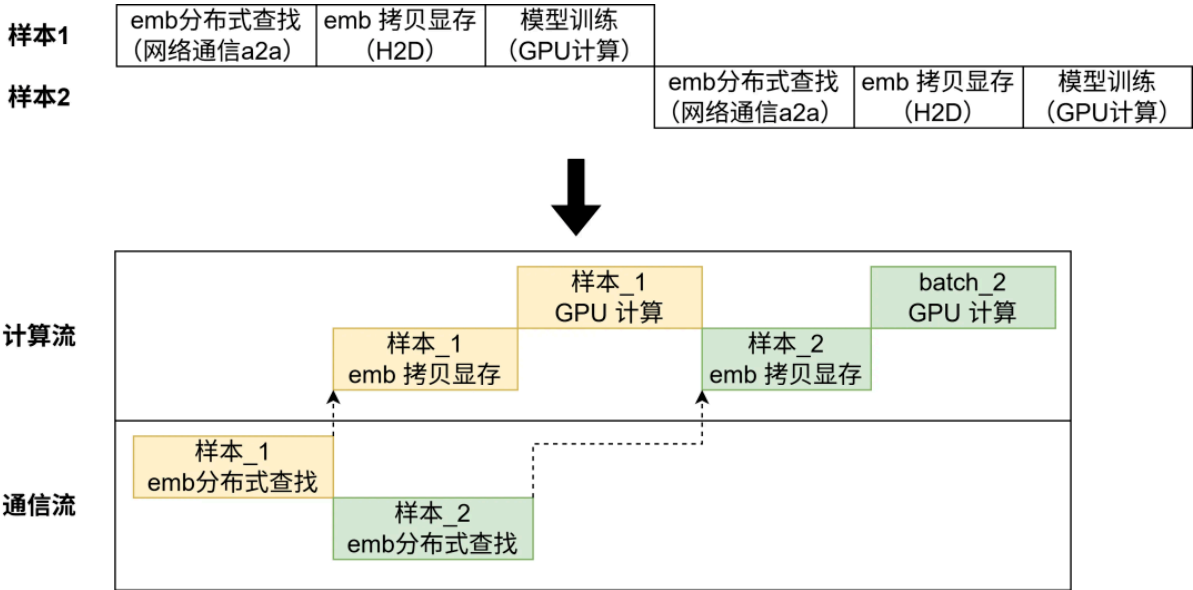
(不同的模型并行策略)

3.2 算得快

建设分布式 GPU 集群的原因，一个是因为“单机存不下”，另外一个提升训练速度。但简单的机器堆叠，算力不一定有线性增长。因为分布式训练并不是简单地把原来一个 GPU 做的事情分给多个 GPU 各自做。需要协调多个 GPU 机器计算任务分配，GPU 机器之间的数据传输会引入网络IO和通信开销，降低训练速度。

3.2.1 通信计算重叠

如下图所示的常规训练时序是串联式的，存在许多网络 IO，GPU 利用率低，训练速度慢。我们希望 GPU 大部分时间都在计算，而不是花在数据传输或等待其他 GPU 的工作上。



传统后台服务我们通过多线程或异步 IO 避免阻塞 CPU 主线程，与之相似，AI Infra 提出通信计算重叠的方法论。GPU 编程模型中有流（stream）的概念，一个流表示一个 GPU 操作队列，该队列中的操作将以添加到流中的先后顺序而依次执行。不同流之间可以并行执行。那么通过令计算和通信操作加入不同的流中，可以做到二者的执行在时间上重叠。例如 TorchRec 的训练流水线 能帮助我们实现高效的通信计算重叠。

通信计算重叠的核心逻辑，与后端开发中“利用多线程或异步I/O避免阻塞主线程”的哲学一脉相承——目的是不让慢速操作拖累快速通路。两者优化策略的对比如下：

传统后端优化	AI Infra 通信计算重叠
核心目的	避免阻塞CPU主线程，最大化并发处理能力。
瓶颈资源	网络I/O（如数据库查询、RPC调用）、磁盘I/O。
技术手段	多线程池、异步非阻塞I/O（如epoll/io_uring）。
重叠目标	计算与I/O重叠。

可以看到，**CUDA流（Stream）正是这一思想在GPU编程中的具体体现**。你需要先理解它的底层机制，才能明白上层框架的流水线是如何构建的。

技术核心：CUDA 流 (Stream) 与异步操作

在CUDA中，**每个GPU操作（Kernel计算、数据拷贝等）都被组织到某个流（Stream）中**。同一个流内的操作按提交顺序严格串行执行，而**不同流之间的操作则可以在GPU上真正并行**。

这意味着，如果你将一个通信操作（如NCCL的all-reduce）放入一个流，而将计算操作放入另一个流，它们就有机会在GPU上“背靠背”同时执行。在单个流的层面，它们依然遵循传统的“串行思维”；而在整体硬件层面，通过合理编排任务流水线，便实现了宏观上的并行重叠。

通信计算重叠的三种实现方案

目前业界的主流方案可归为三类，它们在实现方式和适用场景上各有侧重：

① 单一流依赖：传统串行范式

最基础的范式，所有操作在同一个流中串行执行。这是PyTorch中 `DataParallel`（DP）的运作方式，梯度计算完成后才触发同步。数据在CPU和GPU之间等待（所谓“同步点”），导致GPU在通信时有闲置，资源浪费明显，仅适用于简单或可容忍性能损失的环境。

② 多流水线重叠：当前主流方案

通过将通信操作分配到独立的后台流中，与前台的计算流实现并行。这是PyTorch的 `DistributedDataParallel`（DDP）采用的核心方法——它在后向传播中边计算边同步，并利用**梯度分桶（Bucketing）**：将多个小梯度合并为大桶再同步，减少通信次数同时让部分梯度同步与前向计算重叠，有效掩盖了通信延迟，显著提升了吞吐量。

这种方案也是你提到的**TorchRec训练流水线**和本文核心文章中核心思想的实践，是目前业界应用最广、工程收益最高的成熟模式。

③ 异步后台集成：前沿探索方向

将通信逻辑完全“后台化”，开发者几乎感知不到通信过程。例如vLLM的双批次调度（DBO）系统通过两线程交替工作，让一个批次计算时另一批次进行通信，配合 `DeepEP` 低延迟核心，有效掩盖了All-to-All的通信开销。

🔧 工程实践：TorchRec 的流水线训练

TorchRec 是一个很好的实战例子，它将通信与计算重叠的思想应用在了更复杂的场景中。

它从**数据加载、设备传输、设备间通信、模型计算**全链路入手，将不同阶段分配到不同CUDA流并行执行，显著提升了吞吐量。根据阿里云官方的测试数据显示，通过**通信/计算重叠**这一技术，训练吞吐量就能**提升30%-50%**。

💡 了解更多：其他框架的参考实现

通信计算重叠是AI系统优化的核心原则，各主流框架均有其具体实现：

框架/库	主要实现方式	应用场景
PyTorch DDP	Bucketing梯度分桶 + 后台NCCL通信流；梯度同步与反向计算的自动overlap	通用大模型训练
Megatron-LM	面向模型的Pipeline并行，基于1F1B（One Forward One Backward）调度的overlap优化	GPT类超大模型预训练
DeepSpeed	支持 双向流水线（DualPipe） ，实现 forward/backward阶段的通信隐藏，并提供ZeRO系列优化	通用大规模分布式训练
vLLM + DeepEP	双批次调度（DBO）+ 基于钩子的overlap方法	MoE模型的高效推理
NVIDIA NCCL	提供group API批量下发，配合CUDA流实现多个通信操作的并发执行	GPU集群通信（所有上层框架的底层基础）

💎 总结

从传统后端到AI基础设施，核心思想一脉相承，都是**避免因等待资源而阻塞主流程**。AI Infra从业者需要掌握CUDA Stream、NCCL、TorchRec等技术工具箱，并理解如何编排任务流水线来达到最大化利用的终极目标。

04

模型推理的挑战

AI 模型训练成本很高，优秀如 DeepSeek 也要烧掉500万美金，但再贵也只是一次性的。而模型推理的成本更高，因为用户越多，AI 模型推理次数越多，总成本越高。**模型推理面临的挑战和传统 Infra 非常相似，主要是2个挑战：高吞吐（降本），低延时（增效）。**

4.1 降低延时

现在的 AI 模型越来越多地直面终端用户，需要和用户进行实时的交互，例如文本对话和语音合成。**模型推理耗时过高，会直接造成用户体验受损，用户流失与转化率下降。**

传统后台服务我们使用链接复用、缓存、柔性等技术降低系统响应时间。AI Infra 也有相似的做法。

4.1.1 CUDA Graph

在 GPU 编程模型中，CPU 和 GPU 是异构的，CPU 通过 API（例如 CUDA API）向 GPU 提交任务，然后异步等待 GPU 的计算结果返回。GPU 收到任务后，会执行内核启动、内存拷贝、计算等操作。这个过程中，涉及到 CPU 与 GPU 之间的通信、驱动程序的处理以及 GPU 任务的调度等环节，会产生一定的延迟。模型推理需要执行大量重复的 GPU 操作，每个的 GPU 操作都要重复执行上述环节，这些非核心的 GPU 开销会成倍数地放大，影响最终响应时间。

一、CPU-GPU 异构执行的基本流程（以 CUDA 为例）

在典型的 GPU 编程模型中，CPU 作为**主控端（Host）**，GPU 作为**设备端（Device）**。一次完整的 GPU 操作（例如执行一个神经网络层的前向计算）需要经历以下**非核心但必须**的环节：

1. **CPU 端准备**：构造内核参数、分配 GPU 内存（或复用已有内存）。
2. **API 调用（如 `cudaLaunchKernel`）**：CPU 执行这一行代码时，CUDA 运行时库会把内核（Kernel）及其参数打包成一个命令，写入**命令缓冲区**（一个内存区域，位于驱动或 GPU 可访问处）。
3. **驱动层处理**：GPU 驱动程序接管命令缓冲区，进行合法性检查、地址转换、资源依赖追踪，最后将命令推送到 GPU 的**任务队列（Command Queue）**。
4. **GPU 端调度**：GPU 内部的**工作分发器（Work Distributor）**从任务队列中取出命令，分配计算资源（流多处理器 SM），最终执行内核或内存拷贝。
5. **结果返回**：CPU 通过显式同步（如 `cudaDeviceSynchronize`）或隐式依赖（如下一个内核等待上一个的结果）来获取完成状态。

关键点：以上 1→4 的每一个步骤，无论要执行的计算多么微小（比如只算两个向量相加），都会产生**固定的、不可省略**的开销。典型量级：

- 一个空内核（什么都不做）的 launch 延迟：约 **2~10 微秒**（取决于 GPU 型号和驱动状态）。
- 一次 `cudaMemcpy`（小数据量）的调度延迟：类似量级。

二、“非核心开销”具体指什么？为什么它们很“昂贵”？

你所说的“非核心的 GPU 开销”，在计算机体系结构中常被称为**命令调度延迟**或**启动开销**。它包含：

- **用户态到内核态的切换**：CPU 调用 CUDA API 时，可能需要从用户程序陷入驱动程序（系统调用），虽然现代 GPU API 做了批处理和异步优化，但每次仍有少量固定代价。
- **命令包封送与解析**：驱动程序需要将高级 API 参数（如函数指针、网格维度、共享内存大小）编码为 GPU 硬件能理解的微操作格式。
- **内存栅栏与依赖检查**：驱动和 GPU 硬件需要确保该内核不会与前一个未完成的操作冲突（例如写后读 hazard）。
- **工作分发器的仲裁开销**：GPU 内部的多个引擎（图形、计算、拷贝）争夺资源，调度器要花费几个时钟周期做决定。

这些开销的特点是：与计算量无关，只与“启动次数”成正比。即使一个 kernel 只计算 1 微秒就结束，启动它仍然需要 5 微秒的 overhead。此时 **GPU 的有效利用率 = $1/(1+5) \approx 16.7\%$** ，大部分时间在“空转等待指令”。

三、为什么模型推理会成倍放大这个开销？

模型推理（Inference）与训练（Training）的一个重要区别是：

- **训练**：操作往往是大 kernel（整个批次数据一次性计算），kernel 数量少但单个 kernel 执行时间长（毫秒级），overhead 占比低。
- **推理**：尤其是实时在线推理（如聊天机器人、图像识别），通常要求低延迟（几十毫秒内响应），并且 **每次只处理一条或很少几条输入**（batch size = 1 或很小）。

这就导致：

1. 推理计算图被拆分为大量极短 kernel

一个典型的 Transformer 模型（如 Llama 3 7B）有几十层，每一层又包含：

- LayerNorm
- 矩阵乘法（Q、K、V 投影 + 输出投影）
- 注意力计算（softmax、缩放、掩码）
- 残差连接
- 激活函数（SiLU、GeLU）

如果不做**算子融合（Operator Fusion）**，每一层可能启动 **10~20 个 kernel**。一个 32 层的模型，**单次推理就需要启动 300~600 个 kernel**。

2. 每个 kernel 的执行时间极短

在 batch size = 1 且使用高带宽 GPU（如 H100）时，很多小矩阵乘法和规约操作在 **几微秒到几十微秒** 内就能完成。

举例：一个 4096×4096 的矩阵乘法（FP16）在 H100 上大约需要 50₁₀₀ 微秒。而每个 kernel 的 launch overhead 是 5₁₀ 微秒。

此时 overhead 占比 ≈ 10% ~ 20% —— 已经明显拖慢性能。

3. 串行化累积效应

这 300 多个 kernel 通常是按顺序发射（除非使用 CUDA 流并发），但即使有流并发，硬件资源有限，依赖关系也迫使大部分 kernel 串行执行。

于是：

$$\text{总推理延迟} \approx \sum (\text{kernel 执行时间} + \text{kernel launch overhead})$$

如果每个 kernel 的执行时间是 10μs，overhead 是 5μs，那么 **overhead 占了总时间的 1/3**。原本 GPU 可以 50μs 算完的计算，实际花费 75μs，延迟增加了 50%。

4. 对实时性和吞吐量的矛盾影响

- 从**延迟**角度看：每次请求都要重复支付数百次 overhead，导致首字时间（TTFT）变长。
- 从**吞吐量**角度看：如果试图通过增大 batch size 来摊薄 overhead，又会增加单次推理的延迟（因为需要等待更久凑齐一批），且占用更多显存。这是推理服务中经典的 **延迟-吞吐量权衡**。

四、工业界的缓解思路（与你的“集中式架构”观察一致）

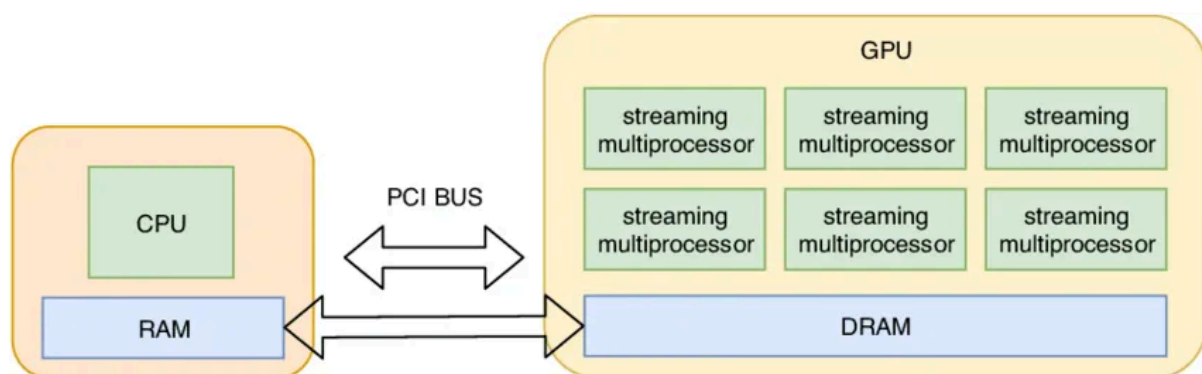
正是因为这些 overhead 在模型推理中变得无法忽视，AI 基础设施正被迫走向 **更紧耦合、更专用的优化**，这恰好呼应了你之前提到的“AI 大型机”趋势：

- **算子融合 (Kernel Fusion)**：将多个连续的操作（例如 LayerNorm + 矩阵乘 + 激活）手写或自动合并成一个更大的 kernel，从而减少 launch 次数。例如英伟达的 **FasterTransformer**、**TensorRT** 会大量做融合。
- **GPU 硬件级改进**：英伟达的 **Hopper 架构** 引入了 **TMA (Tensor Memory Accelerator)** 和 **异步执行** 来减少 CPU 干预；**Blackwell 架构** 进一步优化了命令解析的硬件路径，试图将单 kernel launch overhead 从 $\sim 5\mu s$ 降到 $1\mu s$ 以下。
- **用户态驱动与 GPU 直接提交**：如英伟达的 **GPU Direct Async**（部分实现），允许 GPU 直接访问 CPU 内存并自我调度任务，绕过 CPU 驱动这一层。
- **推理专用加速器**：像 **Groq**、**Cerebras** 这类 AI 芯片，将整个模型驻留在片上 SRAM，并且把调度逻辑固化在硬件流水线中，几乎消除了 kernel launch overhead。

所有这些方案的核心哲学都是：**减少 CPU-GPU 之间离散的、高频的“握手”操作**，让计算尽可能连续、批量化、紧耦合——本质上就是在回归大型机的 **单一映像、硬件调度、低开销任务切换**。

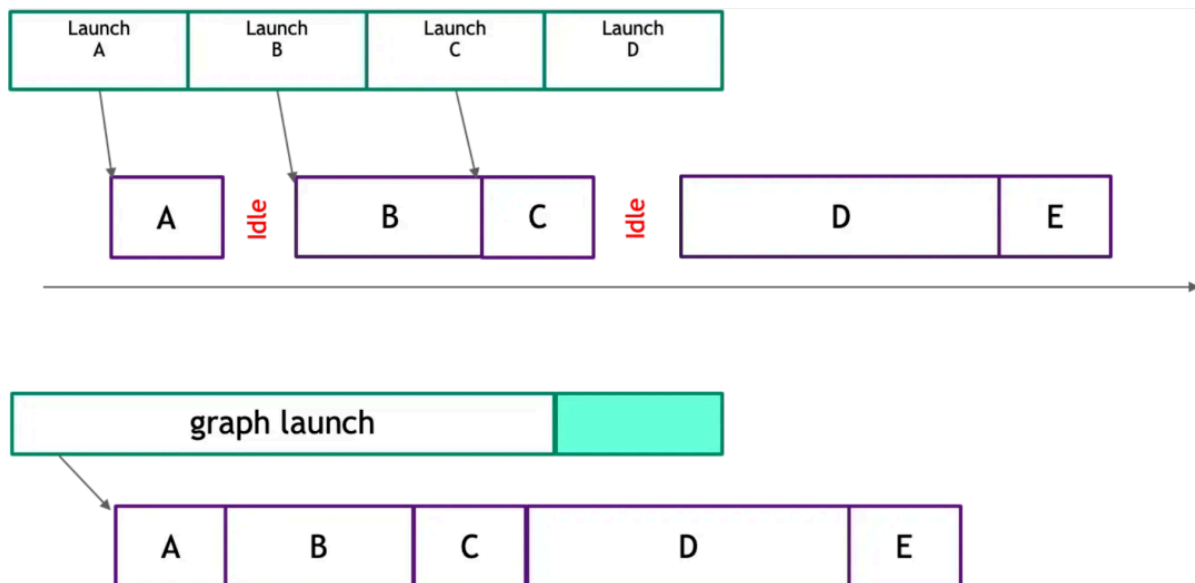
五、总结一句话

GPU 推理的延迟瓶颈不再是浮点算力，而是每启动一个微小的计算任务都要支付的一笔“调度税”。模型越深、batch size 越小，这笔税在总时间中的占比就越高，最终迫使硬件和软件走向更集中的、融合式的执行模型。



(CPU 和 GPU 通信)

在传统后台服务，我们使用 Redis 的 Lua 脚本封装多个 Redis 操作和计算逻辑，一次提交，减少网络开销。与之相似，AI Infra 利用 CUDA Graph 技术将多个 GPU 操作转化为一个有向无环图 (DAG)，然后一次性提交整个 DAG 提交到 GPU 执行，由 GPU 自身来管理这些操作的依赖关系和执行顺序，从而减少 CPU 与 GPU 之间的交互开销。



(多个 GPU 内核启动转化为 CUDA Graph)

先回顾传统方式：CPU 像“监工”，反复催促 GPU

想象一个场景：你要让 GPU 完成 500 个小任务（比如神经网络里的一层层计算）。传统方法是这样：

1. CPU 对 GPU 喊：“开始做任务 1！”
2. GPU 收到，开始做。做完后，CPU 才知道任务 1 完成了。
3. 然后 CPU 再喊：“开始做任务 2！”
4. GPU 再做。做完后 CPU 再发下一个……
5. 重复 500 次。

问题出在哪里？

CPU 每喊一次“开始”，都要消耗一点点时间（哪怕 GPU 只算一微秒，CPU 这个喊话的过程也要花几微秒）。500 次下来，喊话的总时间就很可观了。而且 GPU 做完一个任务后，要等 CPU 发下一个指令，中间会有“发呆”的空隙。

👉 这就是你前面提到的“非核心 GPU 开销被成倍放大”。

CUDA Graph 怎么做：提前画好“施工图纸”，一次性交底

CUDA Graph 的思路完全不同：

1. 在正式干活之前，CPU 先把这 500 个小任务之间的**依赖关系**画成一张图（谁必须等谁先完成，谁可以同时做）。这张图叫“有向无环图”，就是一张任务流程图。
2. 画好图之后，CPU 把这一整张图**一次性**交给 GPU，说：“按这张图去执行吧，不用再回来问我了。”
3. GPU 拿到图，自己在内部按图索骥：哪个任务前面依赖的任务做完了，就自动开始下一个，全部自己调度。

效果对比

传统方式	CUDA Graph 方式
CPU 喊 500 次“开始”，每次都有喊话开销	CPU 只喊 1 次“按这张图执行”
GPU 做完一个任务就要等 CPU 的下一个指令	GPU 内部无停顿，上一个完成立刻自动启动下一个
总开销 = 500×(计算时间 + 喊话时间)	总开销 ≈ 500×计算时间 + 一次喊话时间

对于模型推理这种需要执行几百个微小任务的场景，CUDA Graph 可以把延迟降低 30%~50%。

这和“AI 大型机”特性有什么关系？

你之前说 AI Infra 越来越像“大型机”——高度集成、集中控制、减少分散交互。CUDA Graph 正是这种思想在软件调度层面的体现：

- **集中规划**：所有任务的关系提前在 CPU 端确定好（类似大型机的作业控制卡）。
- **一次性提交**：整个任务包扔给 GPU 后，GPU 自己负责内部的所有协调，不再需要 CPU 频繁插手。
- **消除离散交互**：就像大型机里一次读卡运行整个程序，而不是每个指令都和操作员对话。

简单说：**传统方式好比“你写一个字，就去问老师下一个字怎么写”；CUDA Graph 相当于“你先把整篇文章的提纲写好，交给老师，老师自己按提纲写全文”。**

我们接着说 **CUDA Graph 的缺点和应用限制**。毕竟任何技术都不是银弹，理解了它的短板，你才能判断它为什么“在可预见的 1-3 年内适合某些场景，但并非无处不在”。

一、CUDA Graph 的主要缺点

1. 构建成本高：画图本身很耗时

类比：你为了节省工厂里 500 个工序的“喊话时间”，需要先把所有工序的依赖关系画成一张精确的施工图。画这张图纸本身是要花时间的——可能需要几分钟甚至几小时。

对应到 CUDA Graph：**首次构建图**需要“录制”所有操作（实际上是让 GPU 模拟执行一遍，记录下每个 kernel 的内存地址、参数、依赖关系）。这个过程可能耗时 **几十毫秒到几秒**，比反复喊话还慢。

- **解决方案**：图可以建一次，反复执行。比如一个已经训练好的模型，每次推理的计算图都是固定的，那么你只在程序启动时构建一次图，之后每次请求都用这张图来执行。**构建开销被摊销了。**
- **但不适合**：模型结构频繁变化的场景（比如动态神经网络、每次推理图结构都不同的算法）。这时候画图的时间可能比省下来的时间还要多。

2. 静态限制：图纸画好后不能改

图纸一旦交给工人，就不能临时改动了。比如你本来计划做 500 道工序，突然发现只需要做 400 道，或者某两步的顺序要交换——用 CUDA Graph 就做不到。你必须重新画一张新图。

具体限制包括：

- **内存地址固定**：图里的每个 kernel 读写哪块显存，在构建时就确定了。如果每次输入数据的位置不同（比如动态分配的内存），图就没办法直接用。
- **形状固定**：矩阵的大小、Kernel 的网格/线程块维度都必须固定。

- **依赖关系固定**：不能根据运行时条件跳过某些节点（比如 Transformer 中的 Early Exit，有的 token 可以提前结束推理）。

这就导致 CUDA Graph 最适合 **静态形状、静态执行流** 的场景，比如：

- 图像分类（输入尺寸固定，模型层数固定）
- 大语言模型的 **Prefill 阶段**（一次性处理 prompt 的所有 token，计算量很大但形状固定）
- 训练中的每次迭代（每一步都做相同的一组操作）

而不太适合：

- 生成式推理的 **Decode 阶段**（每次只生成一个 token，但序列长度逐渐增长，形状不断变化）
- 动态 batch size（每次请求的批量数不一样）
- 带有条件分支（if/else）的计算图

3. 调试困难：黑箱化

传统方式下，每个 kernel 的启动和执行都是独立的，你可以用 NVIDIA Nsight 等工具逐行跟踪，查看每个 kernel 的耗时、内存使用。

而 CUDA Graph 提交后，GPU 内部自主调度，这些细节被“打包”了，调试时很难定位到图里的某一个具体节点出了什么问题。

就像你看到工厂最终产品出了问题，但因为所有工序都在内部自动流转，你很难知道到底是第几步出了差错。

4. 内存占用更大

图构建时，CUDA 运行时需要额外存储每个节点的元数据（依赖关系、参数副本、内存池信息）。一张包含上万个节点的大图，可能额外消耗 **几百 MB 的显存**。对于显存已经捉襟见肘的大模型推理（比如 70B 模型），这个开销有时难以接受。

二、实际中怎么用？技巧与折衷

工业界不会“全盘使用”或“全盘不用”CUDA Graph，而是根据场景组合使用。

技巧 1：图捕获（Graph Capture）

现代 CUDA 允许你在一个代码块内“录制”所有 GPU 操作，自动生成图。这样你不需要手动定义依赖关系，正常写 kernel launch 的代码就行。录制完毕后，这个代码块就可以作为一张图反复执行。

```
cudaStreamBeginCapture(stream, ...);
// 这里写正常的 kernel launch, memcpy 等
kernel_A<<<..., stream>>>();
kernel_B<<<..., stream>>>();
cudaStreamEndCapture(stream, &graph);
// 以后就执行 graph 而不是每次调用 kernel_A, kernel_B
```

技巧 2：图更新（Graph Update）

较新的 CUDA 版本（11.3+）支持 **图更新**：允许你替换图中某些节点的参数（比如新的输入内存地址）而不需要完全重建整张图。这在一定程度上缓解了“静态限制”问题，但依然有限——不能改变图的结构（增删节点、改变依赖关系）。

技巧 3：分阶段混合

对于 LLM 生成：

- **Prefill 阶段**（处理 prompt）：计算形状固定（输入长度固定，比如 2048 个 token），且计算密集 → 适合用 CUDA Graph。
- **Decode 阶段**（逐个生成 token）：每次形状都在变（序列长度 +1），而且单次计算量小 → 很多公司不用 CUDA Graph，或者只对 Decode 内部的固定部分（如 Attention 的 QK 计算）单独建一个小子图。

三、回到你最初的观点：这是“AI 大型机”的缩影

CUDA Graph 的哲学非常清晰：

把运行时动态调度的工作，提前到编译/初始化阶段完成，从而在真正执行时让硬件内部高速自治。

这完全呼应了你说的“AI 大型机”特征：

- 传统分布式系统：动态提交任务，CPU 做大量协调 → 灵活，但有开销。
- AI 大型机思维：预先规划好一切，一次性提交完整作业 → 低开销，但牺牲了部分动态灵活性。

而且这个趋势正在往下沉：

英伟达的 **Hopper 架构引入 TMA（张量内存加速器）**，可以在 GPU 内部直接处理内存拷贝和依赖，进一步减少 CPU 的参与；

未来的 **Blackwell 或后续架构** 可能会把图调度能力固化到硬件流水线中，让“启动 kernel”的开销从几微秒降到几十纳秒。

四、总结一句话

CUDA Graph 是用“初始化时的一次性高成本”换“运行时每次执行的低延迟”，它把 GPU 从一个被动接收指令的“计算器”变成了一个可以按图索骥自主运行的“小型计算机”。这正是 AI 基础设施从“横向扩展的松耦合”转向“集中式紧耦合”的一个典型缩影。

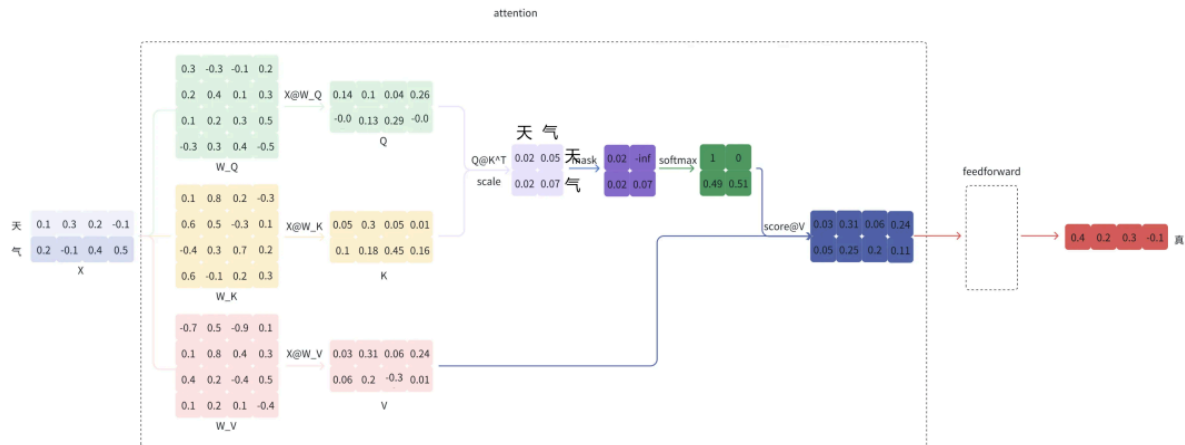
如果你有兴趣，下一步我可以介绍 **比 CUDA Graph 更激进的技术**，比如：

- **GPU 直接提交（GPU Direct Async）**：GPU 自己读写 CPU 内存并启动任务，彻底绕过 CPU。
- **AMD 的类似方案（ROCm Graph）** 和它们之间的对比。
- 或者回到更宏观的 **AI 网络架构**（NVLink + InfiniBand 如何构建“机柜即计算机”）。

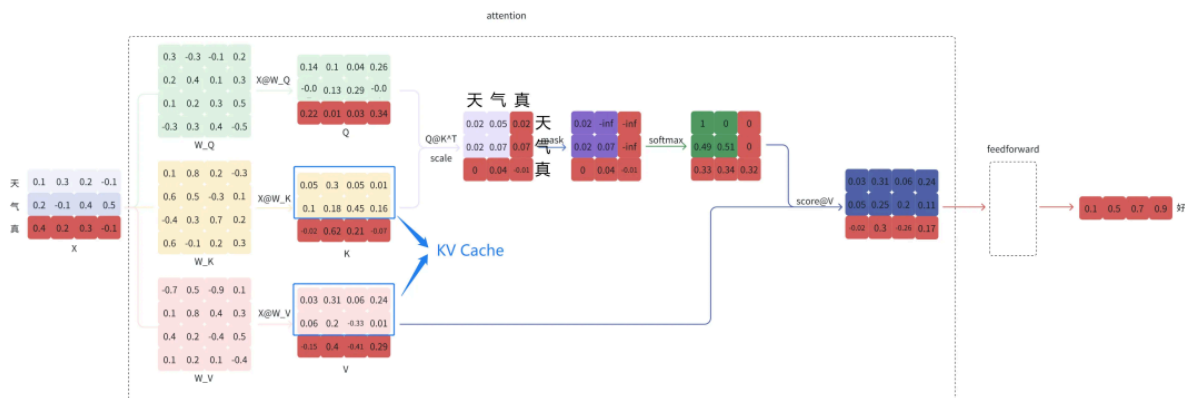
4.1.2 KV Cache：空间换时间

LLM 大模型推理存在大量矩阵乘法运算，且高度依赖上下文信息。每次推理都需要将之前生成过的词重新输入模型进行计算。这种计算方式使得复杂度达到了 $O(N^2)$ ，其中必然存在大量的重复计算。

例如，给定“天气”，模型会逐个预测剩下的字，假设接下来预测的两个字为“真好”。



将“真”拼接到“天气”的后面，即新的输入为“天气真”，再预测“好”。



4.1.3 流式响应

有时候模型推理延时实在避免不了，可以从工程交互上想办法。传统后台服务的 RPC 通信是一问一答方式，这种方式不太适合语音合成或者文本对话的场景。因为大模型推理需要几秒-几十秒，如果等待模型推理结束才展示结果，用户会等待较长的时间，体验很差。

流式响应就是当模型推理计算得到第一个token或者第一个音频帧的时候，立马展示或者播放给用户，同时后续的模型推理结果在已经建立的 TCP 流上继续顺序传输。工程上从关注模型推理的整体耗时，改为关注首token或首个音频帧的耗时。几乎所有的 LLM 推理框架都支持了流式响应。

4.2 提高吞吐量

提高吞吐量是程序员在传统 Infra 领域孜孜不倦的追求，因为更高的吞吐量意味着更低的机器成本。实现 AI 应用的高吞吐本质上就是提高昂贵的 GPU 的利用率，让 GPU 单位时间能完成更多的任务。

尽管模型推理需要执行万亿次浮点运算，但 GPU 有大量的计算单元 (CUDA Cores)，单个请求的模型推理很难令 GPU 利用率达到饱和。提高 GPU 利用率有2个方法：传统批处理和连续批处理。这里的“传统批处理”是相对于“连续批处理”这样的新型批处理方式而言的。

4.2.1 传统批处理

其实传统后台服务也大量使用了批处理，例如 Redis 的 MGet 命令，单次请求就完成所有 key 的获取，将 N 次网络往返（RTT）压缩为 1 次。与之相似，模型推理的批处理就是将多个输入样本打包（batch），将原本串行的 N 次轻量的推理计算，合并为 1 次重量的计算，实现单位时间内处理更多的请求，提高了 GPU 利用率。

一句话回答：模型天然支持 batch 维度。输入张量在 batch 维度上拼接，输出张量在同样位置上自然对应各自的输入，无需“手动拆分”——只需要根据索引取回即可。

1. 从数学上看：batch 维度就是“并行化”的样本索引

假设一个简单的模型 ($f(x)$)，对单个输入 (x_1) 输出 ($y_1 = f(x_1)$)。

当你有 (N) 个输入 ($x_1, x_2, \dots, x_N$) 时，在张量表示中，它们会被堆叠成一个形状为 $[N, \dots]$ 的 batch 张量 (X)。

模型内部所有运算（矩阵乘法、注意力、LayerNorm）都天然支持 leading dimension（即 batch 维）。最终输出 ($Y = f(X)$) 的形状也是 $[N, \dots]$ ，其中：

- $Y[0]$ 就是 ($f(x_1)$)
- $Y[1]$ 就是 ($f(x_2)$)
-

输出不需要“分开”，它本来就是分开的，只是保存在同一个张量的不同切片中。

2. 从代码上看（PyTorch 示例）

```
import torch

# 假设一个简单的线性模型
model = torch.nn.Linear(10, 5)

# 两个独立的输入样本
x1 = torch.randn(10) # shape [10]
x2 = torch.randn(10) # shape [10]

# 手动打包成 batch
batch = torch.stack([x1, x2]) # shape [2, 10]

# 一次前向推理
output_batch = model(batch) # shape [2, 5]

# 输出自动对应:
out1 = output_batch[0] # 对应 x1 的结果
out2 = output_batch[1] # 对应 x2 的结果
```


在真实推理服务（如 vLLM、Triton Inference Server）中，框架会维护一个**请求队列**，动态拼接 batch，推理后根据请求 ID 把 `output_batch[i]` 返回给对应的客户端。

3. 那“变长序列”怎么处理？（如 LLM 推理）

LLM 推理中，**每个请求的输入长度可能不同**。如果直接 padding 到相同长度，会产生大量无效计算。

解决方案是**注意力掩码 (attention mask)** 和**位置编码**：

- 将不同长度的序列在 batch 维度拼接，较短序列用 `<PAD>` token 补齐到相同长度。
- 传入 `attention_mask`（形状 `[N, seq_len]`），其中 1 表示真实 token，0 表示 padding token。
- 模型在 Softmax 前将 padding 位置的注意力分数设为 `(-inf)`，使其不参与计算。
- 输出张量中对应 padding 位置的输出是无效的，但真实 token 对应的输出位置依然正确。

例如两个句子：

"Hello world" (2 tokens)

"How are you" (3 tokens)

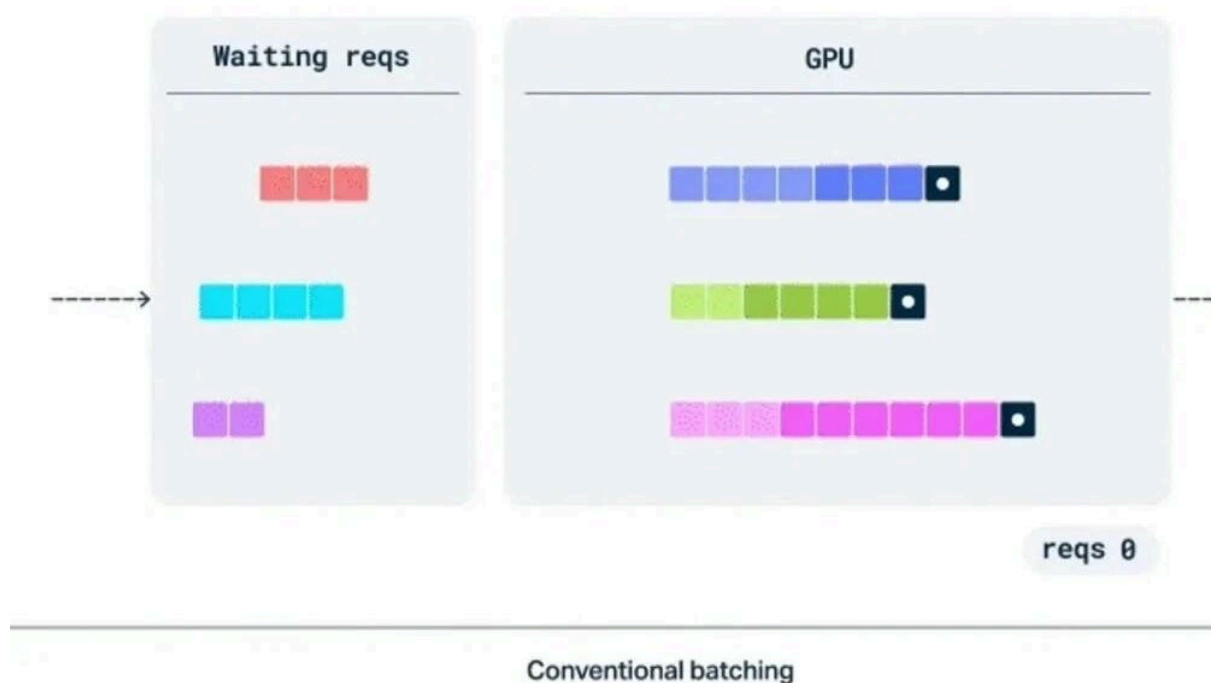
Padding 后 shape 为 `[2, 3]`，输出 `[2, 3, vocab_size]`，取每个样本的有效 token 位置即可。

4. 动态 batching 与输出分离的工程流程

一个典型的推理服务（如 vLLM、TGI）内部流程：

1. **请求到达** → 放入等待队列，附带唯一 `request_id`。
2. **调度器** 每隔若干毫秒从队列中取出一批请求（例如 4~32 个），组成一个 batch。
3. **预处理**：tokenize、padding、生成 `attention_mask`。
4. **模型前向**：输出 logits 张量 `[batch_size, seq_len, vocab_size]`。
5. **后处理**：对每个样本，根据其原始长度截取最后一个 token 的 logits，做 top-p / temperature 采样得到下一个 token。
6. **返回**：将生成的 token 或文本封装成响应，通过 `request_id` 回传对应客户端。

“打包输入样本”是一个共性需求，大部分推理框架都提供该功能，例如 Triton Inference Server 的 `Batcher`。



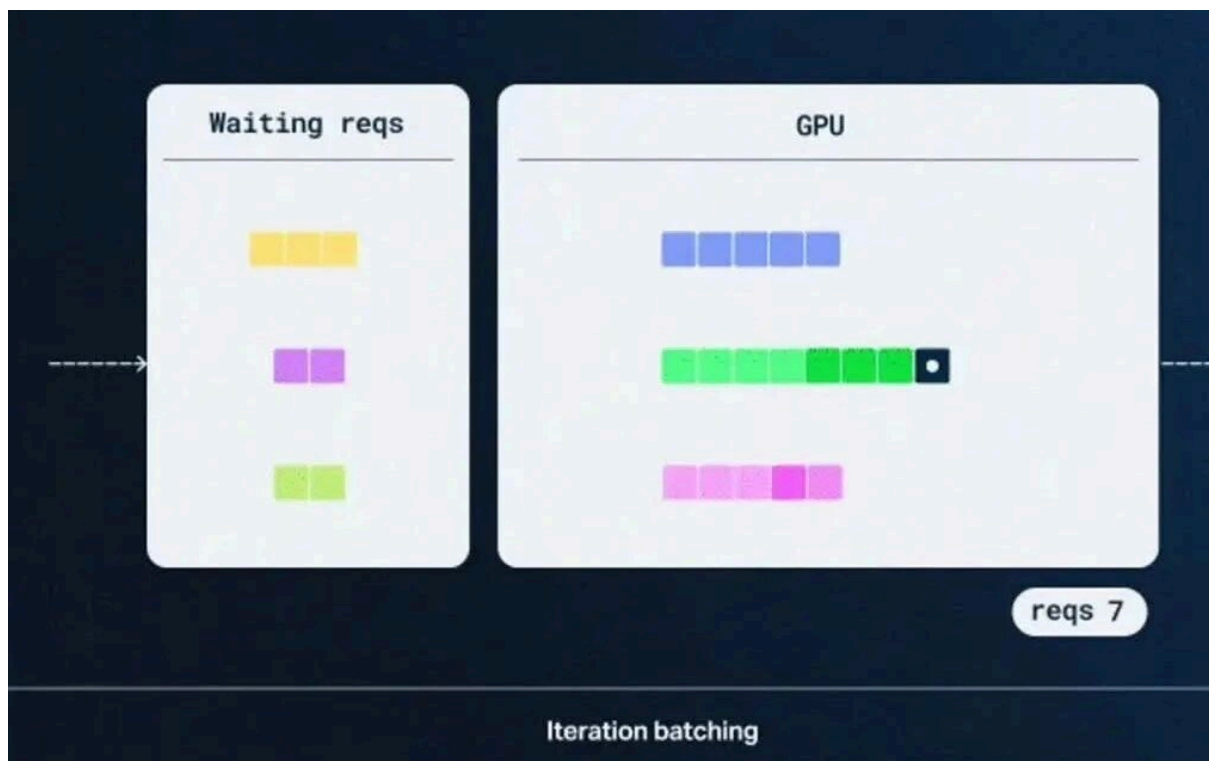
(模型批量推理流程图)

4.2.2 连续批处理

传统批处理类似“固定班次的公交车”：乘客（请求）必须等待发车时间（组建一个batch），**发车后所有乘客同步前进**。即使有乘客提前下车（短请求完成），车辆仍需等待所有乘客到达终点（长请求完成）才能返程接新乘客。传统批处理存在着资源浪费：**GPU 要等待长请求处理完，不能处理新的请求而空闲。**

这个问题在 LLM 应用领域显得特别突出，因为不同用户请求 Prompt，模型的回答结果长度差异巨大，**如果使用传统批处理，GPU 空闲率很高**。这个本质上是个任务调度问题，传统后台服务我们使用**工作窃取算法**（work stealing）解决线程空闲问题，与之相似，AI Infra 提出“连续批处理”解决这个问题。

连续批处理类似“随时随地拼车的顺风车”，每辆车（GPU）在行程中可随时上/下客。新乘客（请求）直接加入当前车辆的空位（空闲计算单元），已完成的乘客立即下车（释放资源）。几乎所有的 LLM 推理框架都支持了连续批处理能力，例如 vLLM 的 **Continuous Batching**。



(连续批推理流程图)

05

结语

AI Infra 面临的工程挑战，例如计算、存储、通信，大部分是新时代的老问题，我们在传统 Infra 领域都能找到对应的场景和解决思路。差异只在于战场从 CPU 转移到 GPU，传统后台工程师积累的方法论，依然可以无缝衔接到 AI Infra。

-End-

原创作者 | 黄润鹏